

Collections and Generics in C#

Engineering Queuing Solutions



Zoran Horvat

CEO at Coding Helmet

@zoranh75 | <https://codinghelmet.com>



Introducing Queues and Stacks

Queue<T>

vs

Stack<T>

Exposes Enqueue() and Dequeue()

Operates in $O(1)$ time and space per item

A.k.a. FIFO (first-in, first-out)

FIFO problems are comparatively rare

Often used as temporary storage

Queues are common in multithreading

Decouple producers from processors

Exposes Push() and Pop()

Operates in $O(1)$ time and space per item

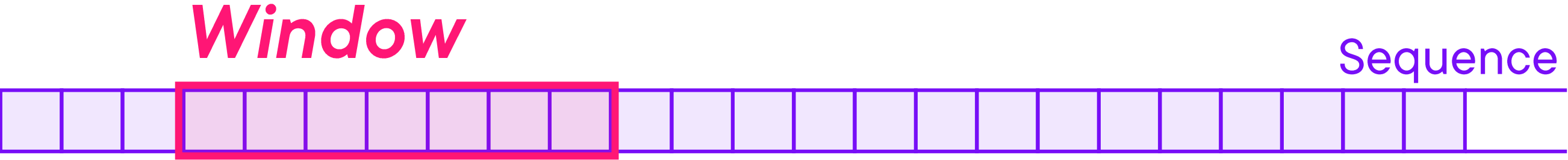
A.k.a. LIFO (last-in, first-out)

LIFO problems are rare *indeed*

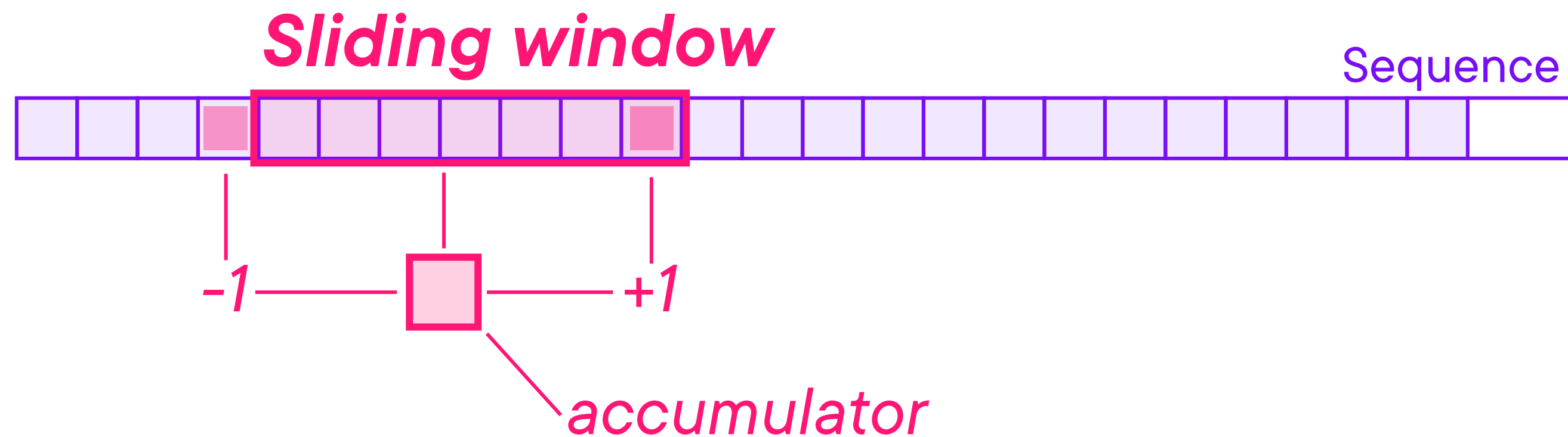
Often used as temporary storage



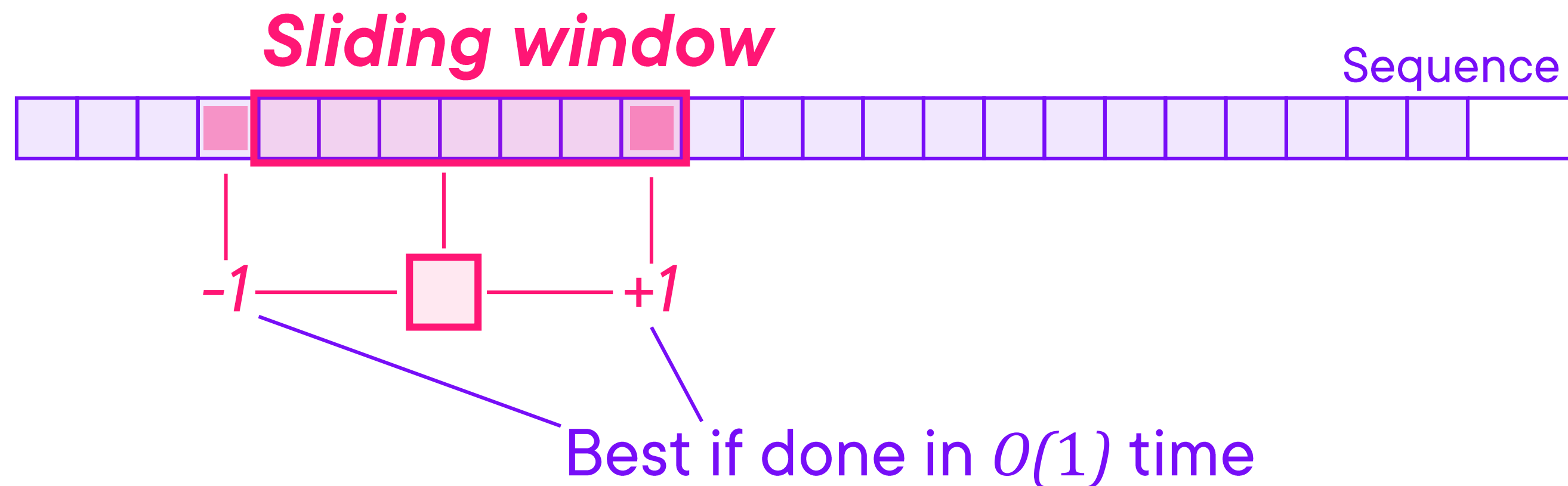
Introducing the Sliding Window Technique

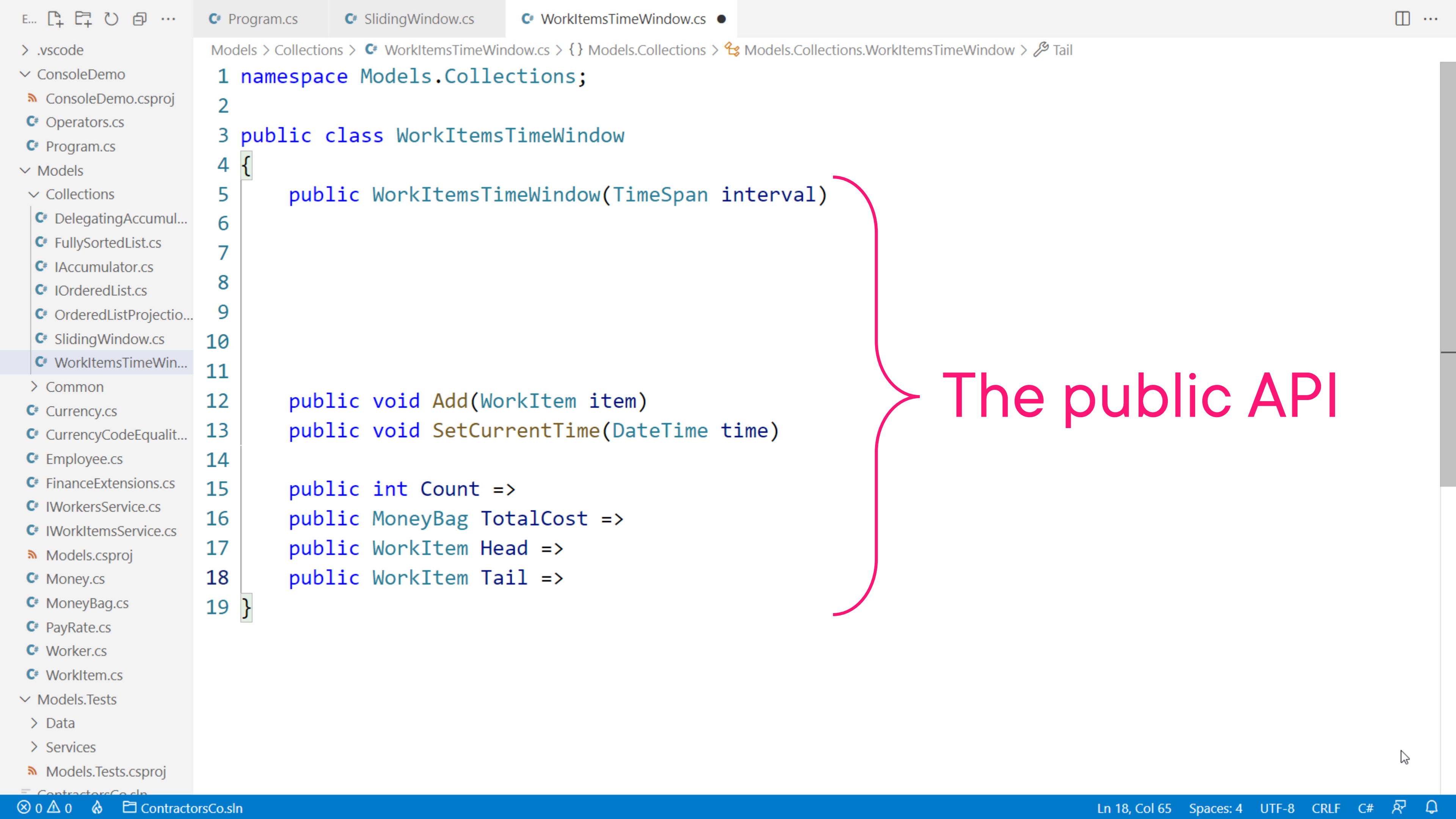


Introducing the Sliding Window Technique



Introducing the Sliding Window Technique

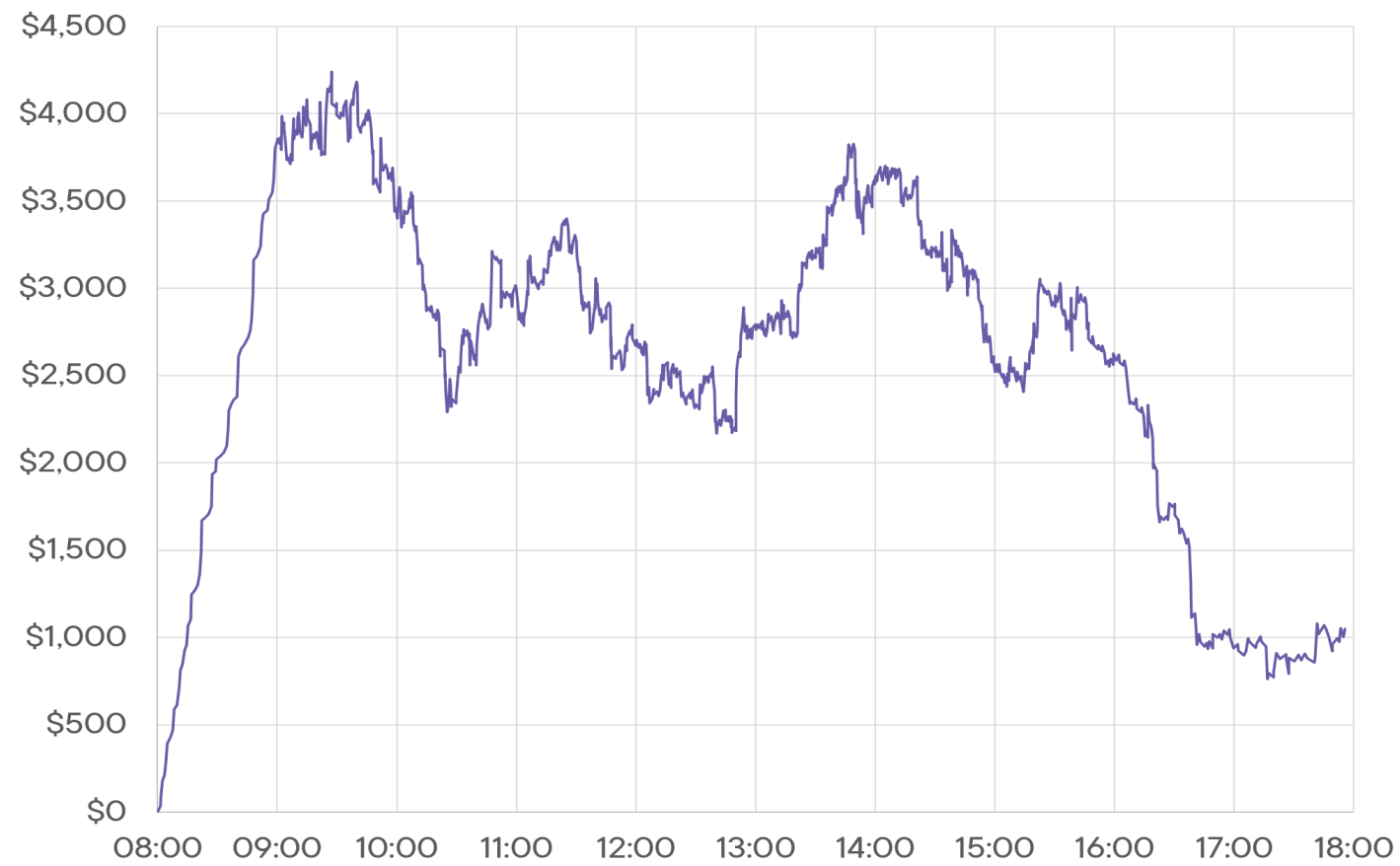




```
1 namespace Models.Collections;
2
3 public class WorkItemsTimeWindow
4 {
5     public WorkItemsTimeWindow(TimeSpan interval)
6
7
8
9
10
11
12     public void Add(WorkItem item)
13     public void SetCurrentTime(DateTime time)
14
15     public int Count =>
16     public MoneyBag TotalCost =>
17     public WorkItem Head =>
18     public WorkItem Tail =>
19 }
```

The public API





Customer's request:

At any moment,
display the highest cost
of any work item
assigned during the last hour



Analyzing the Request



We have the working sliding window algorithm

- We shall apply it to solve the maximum problem



There is no such accumulator that can quickly report a maximum

- When current maximum is removed, we do not know its successor



The sliding window algorithm is fine

- The accumulators are not





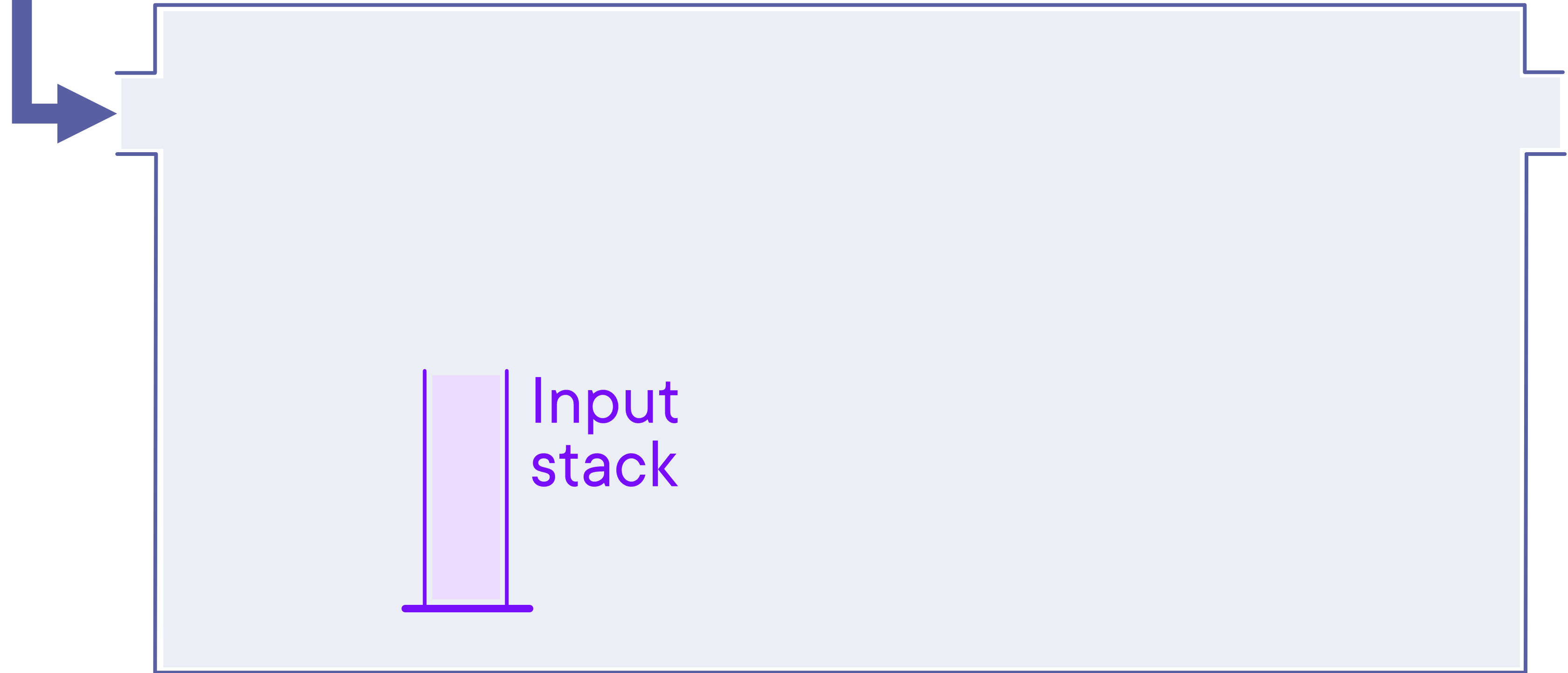
Constructing a queue



Constructing a queue using two stacks

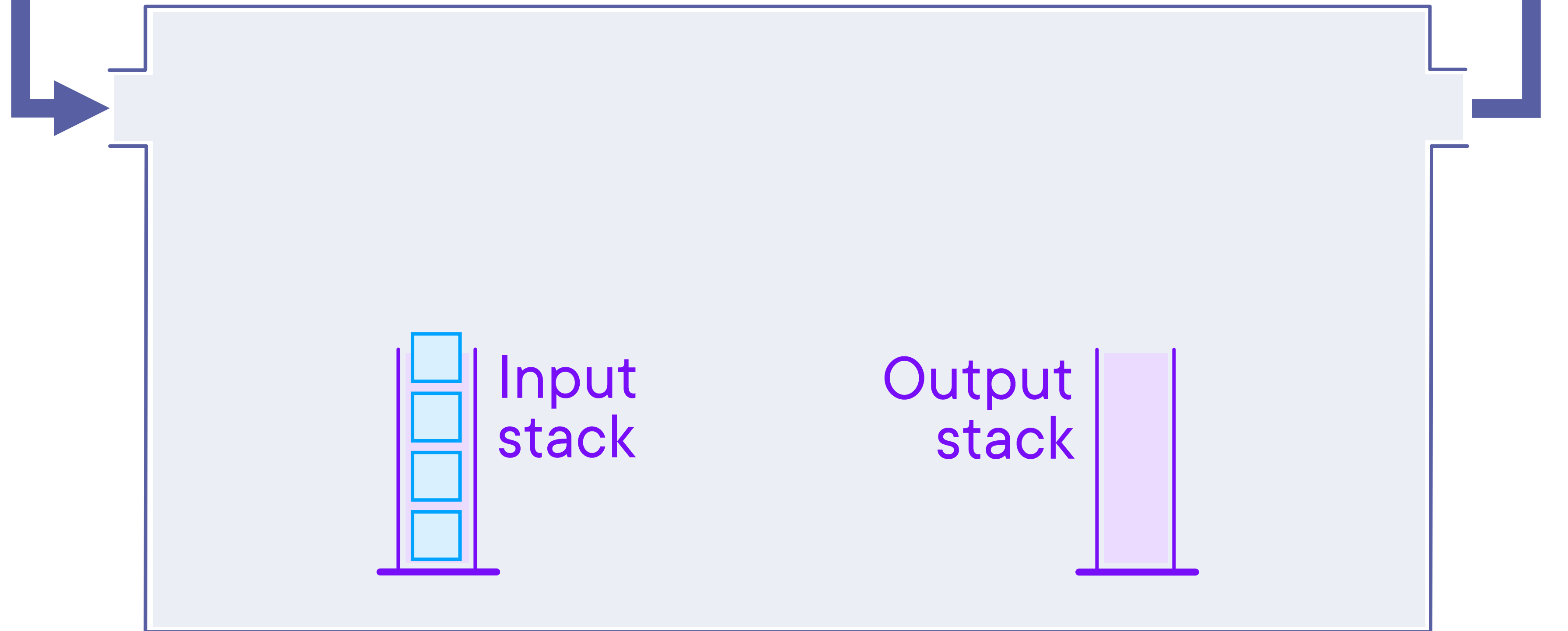


Enqueue



Enqueue

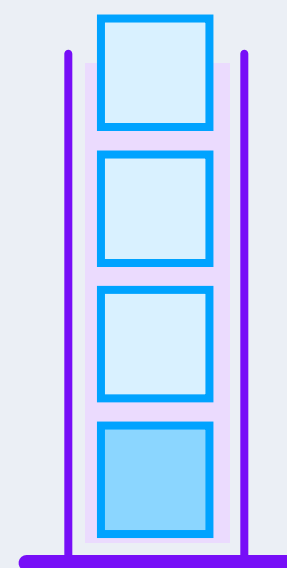
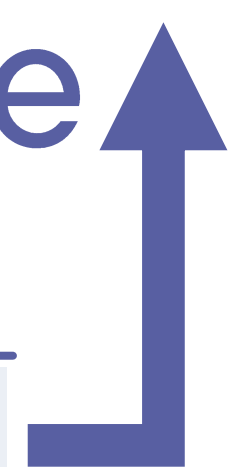
Dequeue



Enqueue



Dequeue



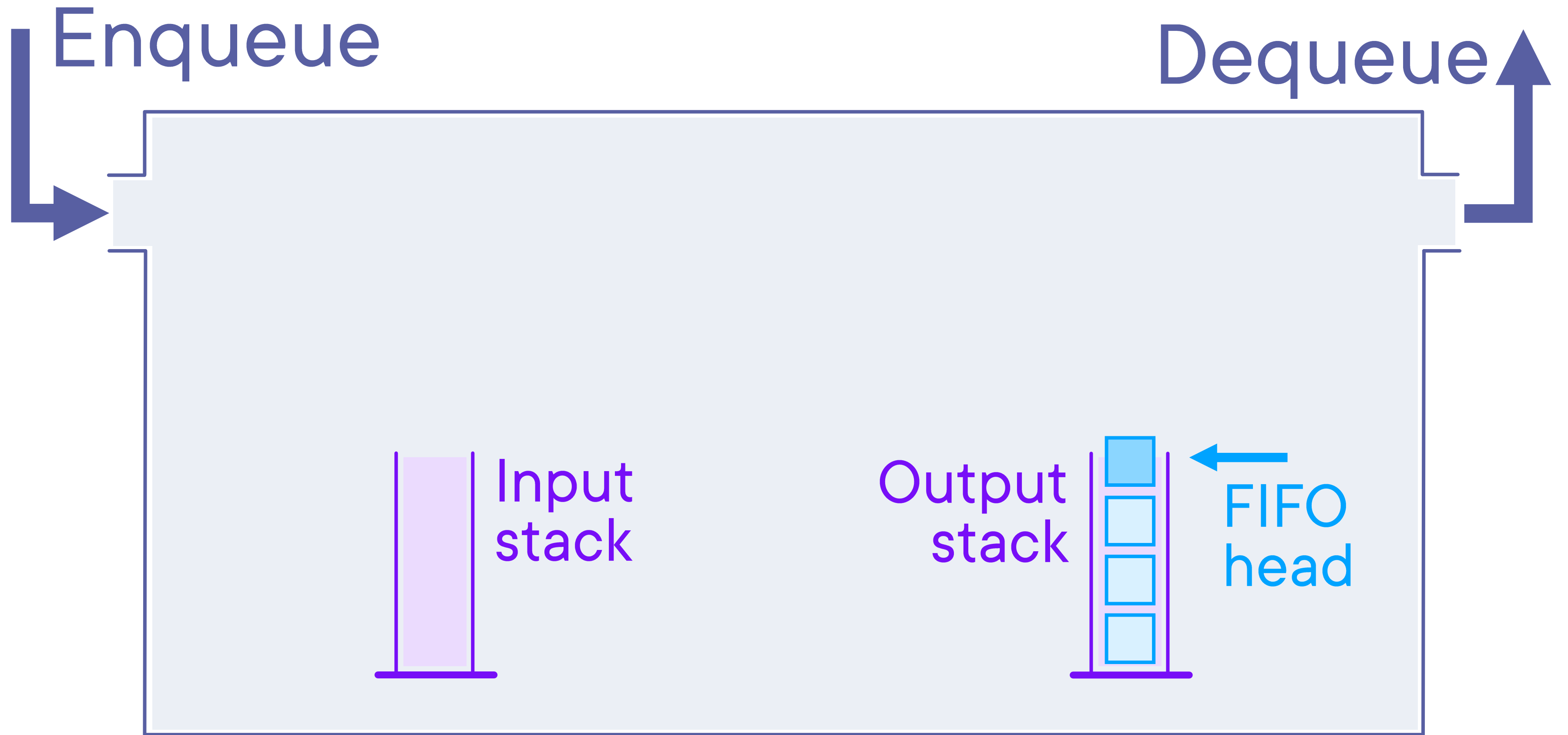
Input
stack

← FIFO head

Output
stack



Enqueue



Dequeue

Input
stack

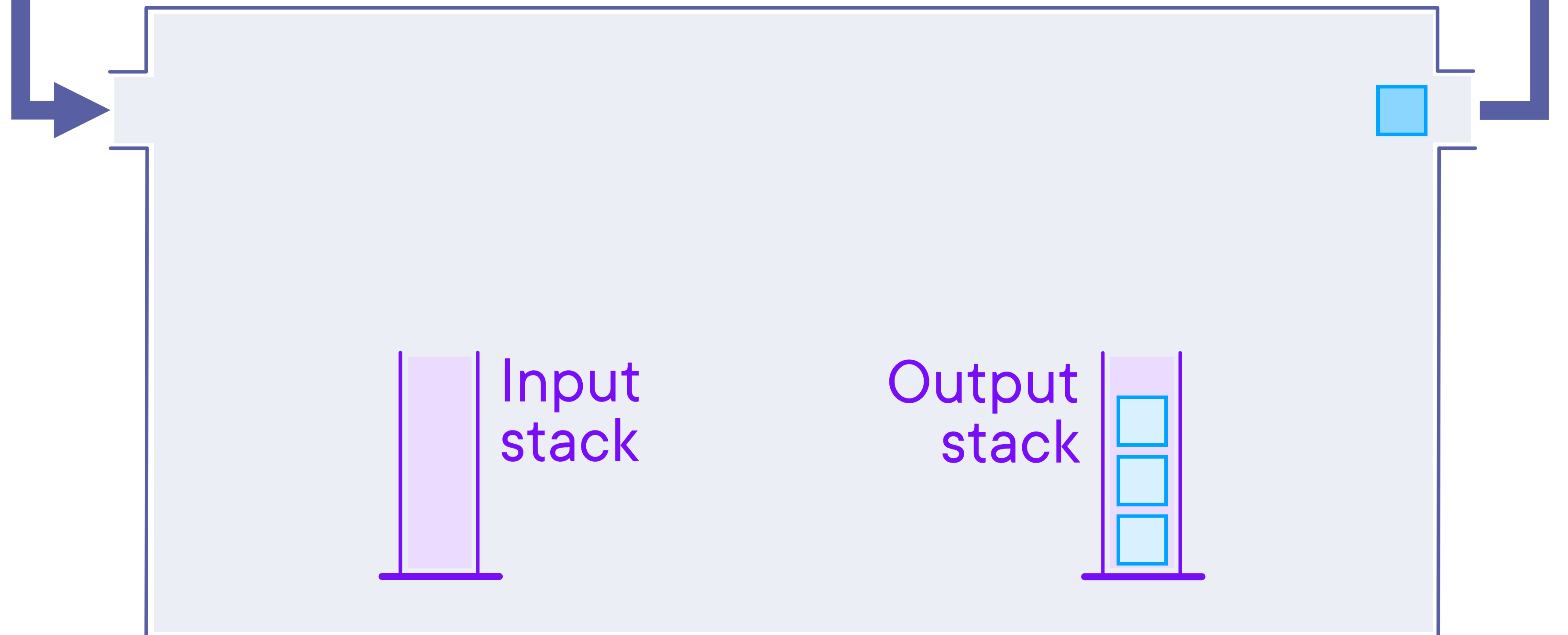
Output
stack

FIFO
head



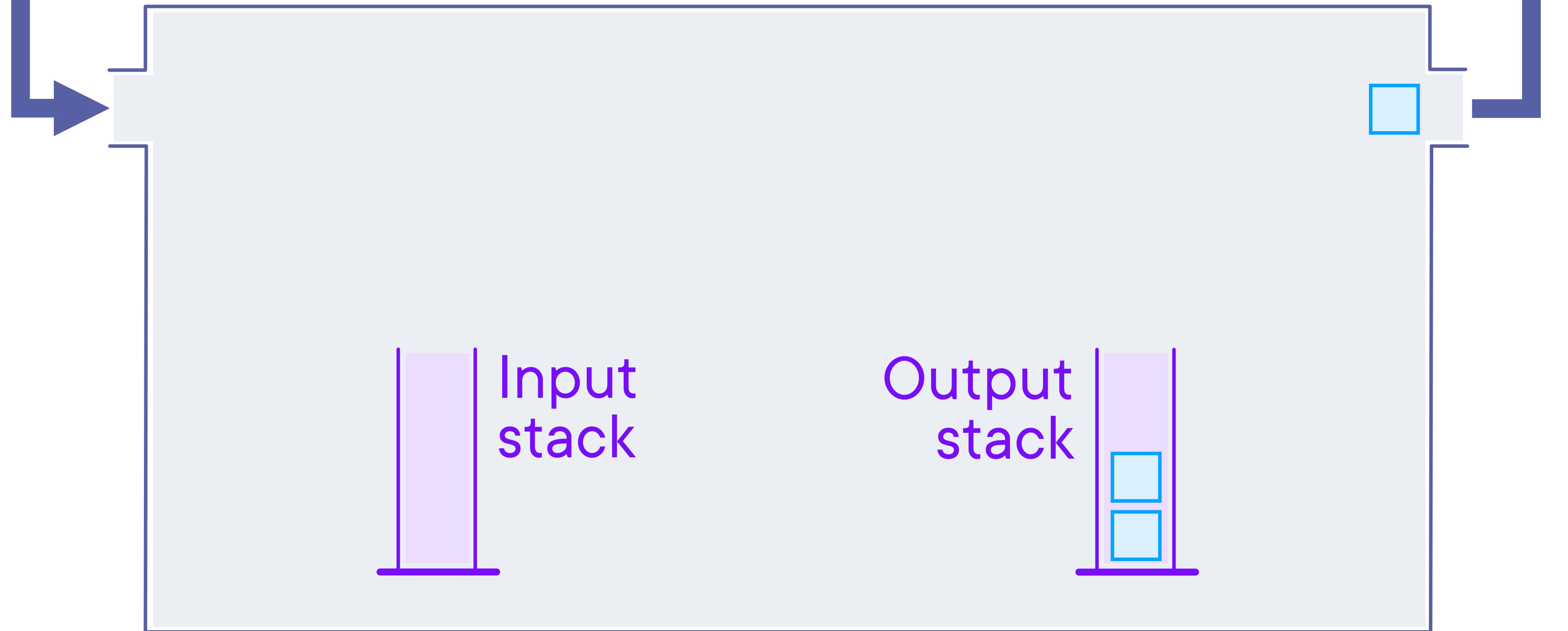
Enqueue

Dequeue



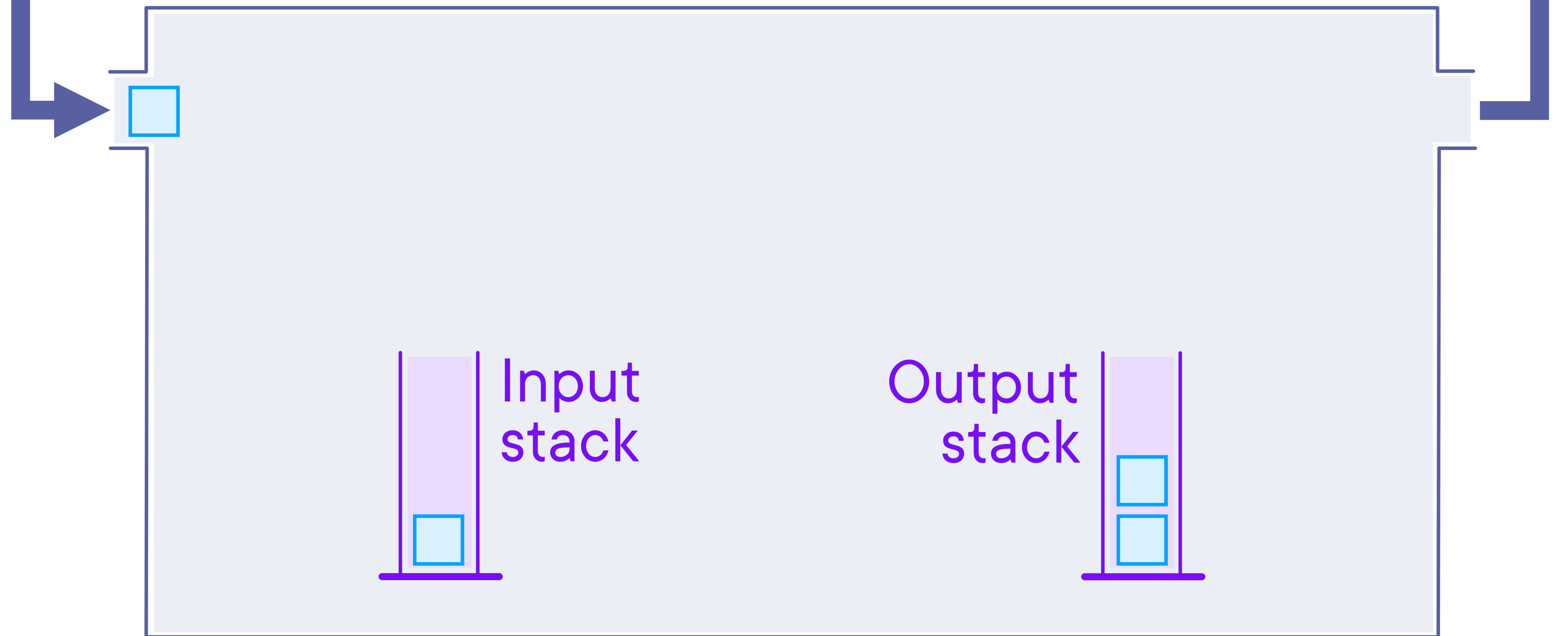
Enqueue

Dequeue



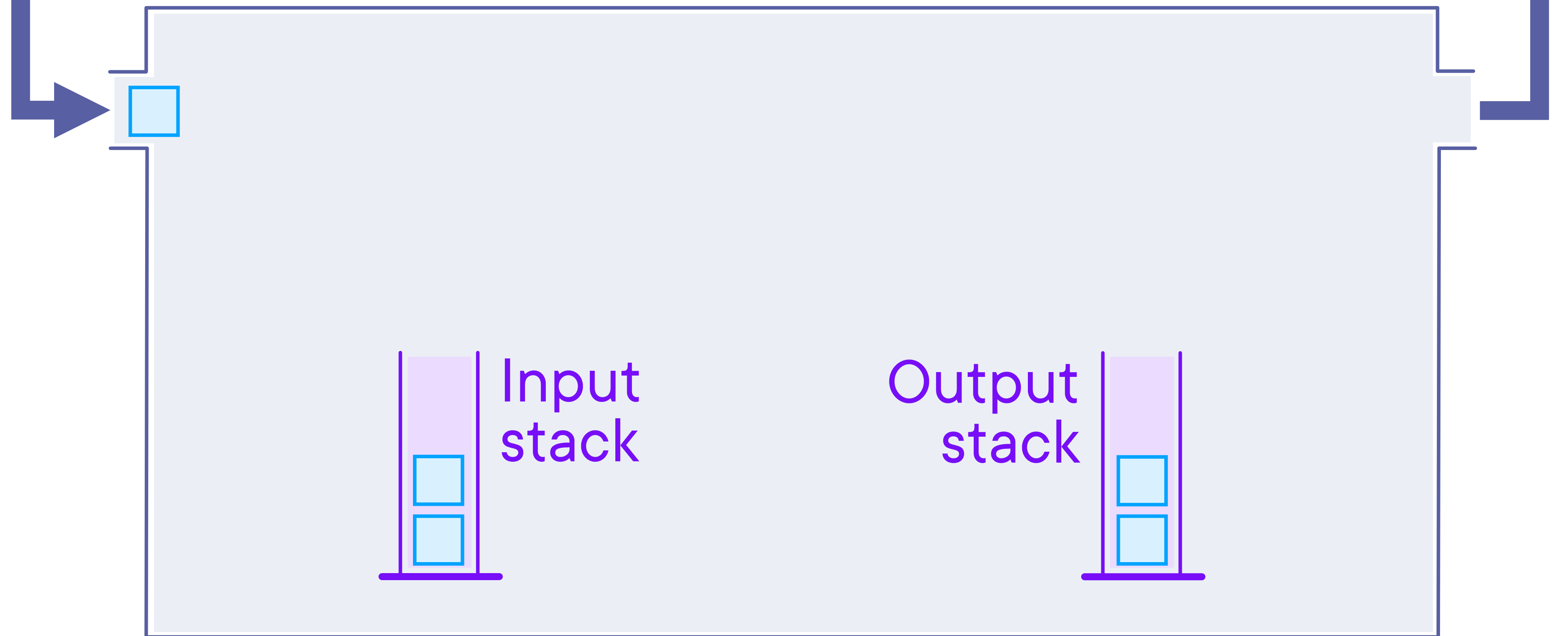
Enqueue

Dequeue

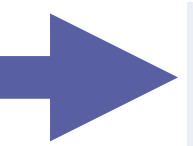


Enqueue

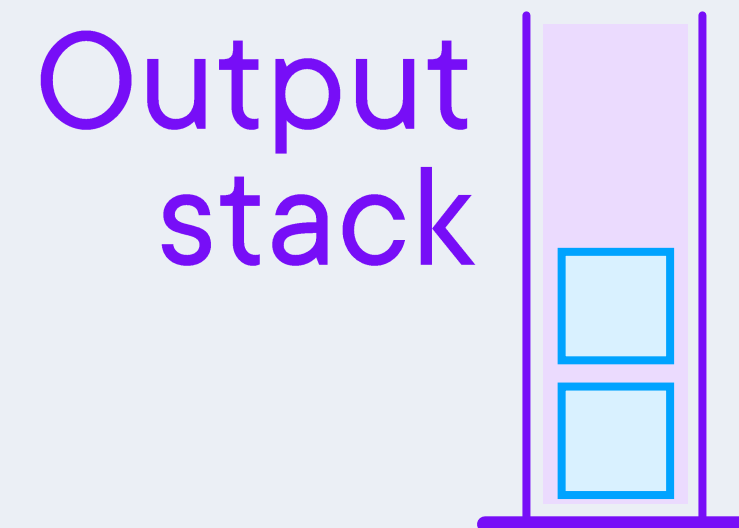
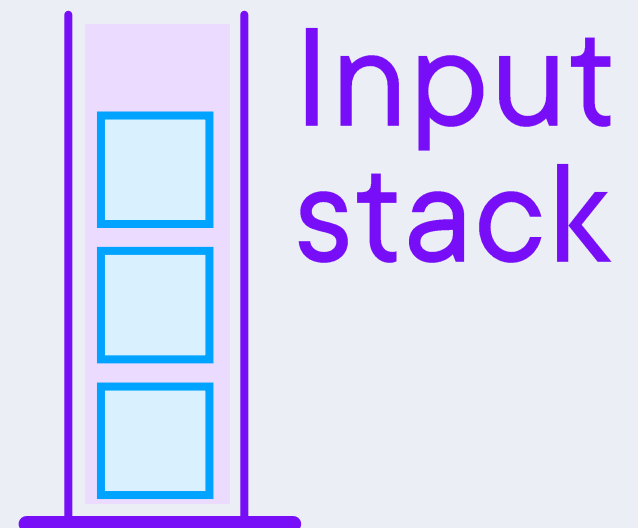
Dequeue



Enqueue

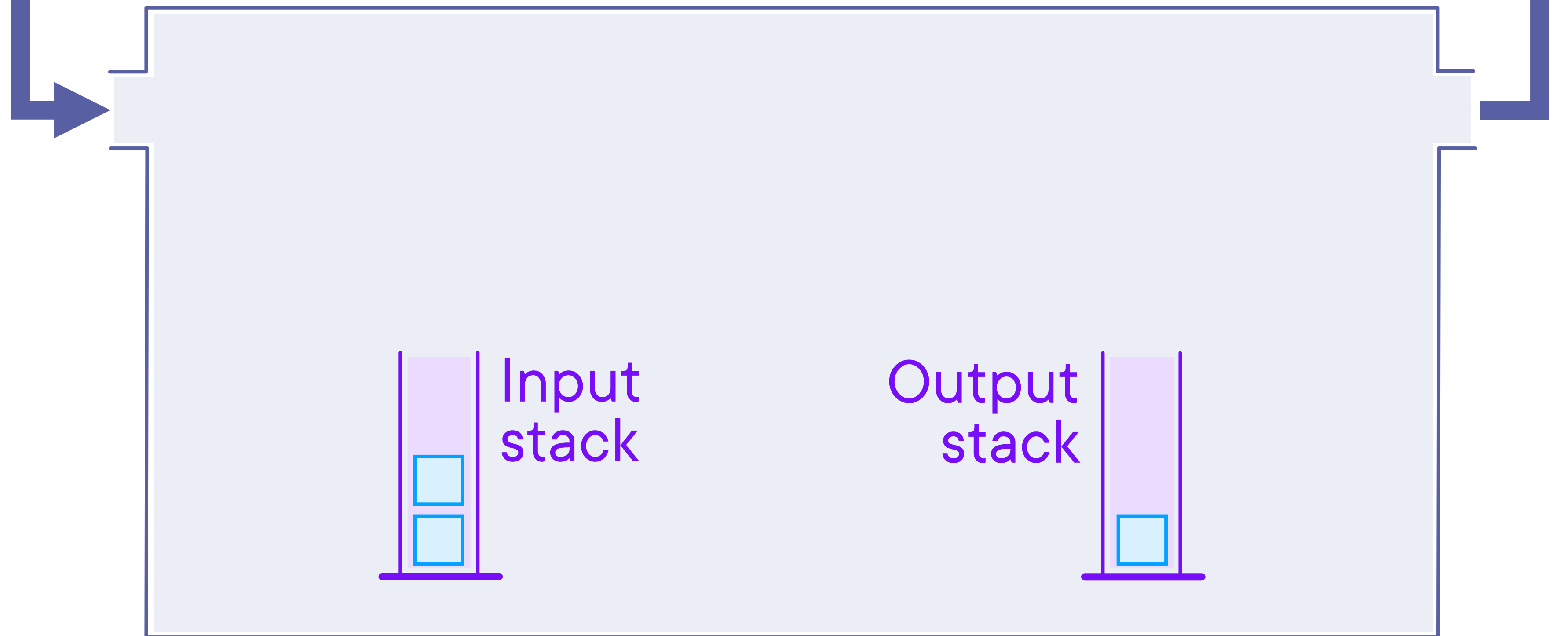


Dequeue



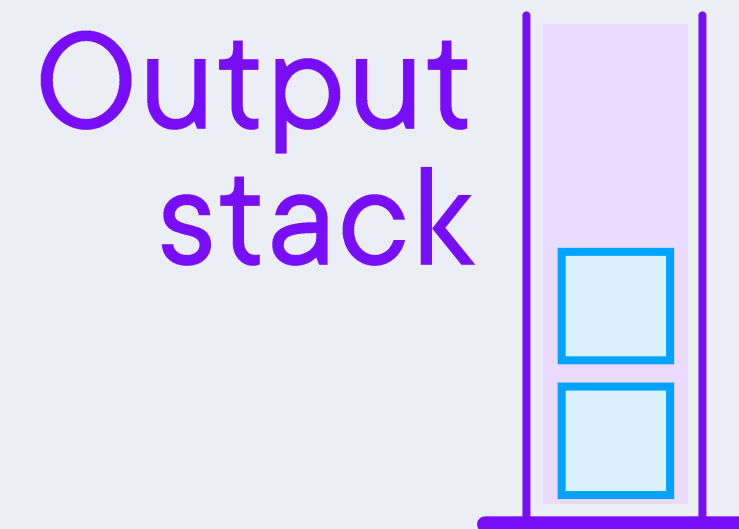
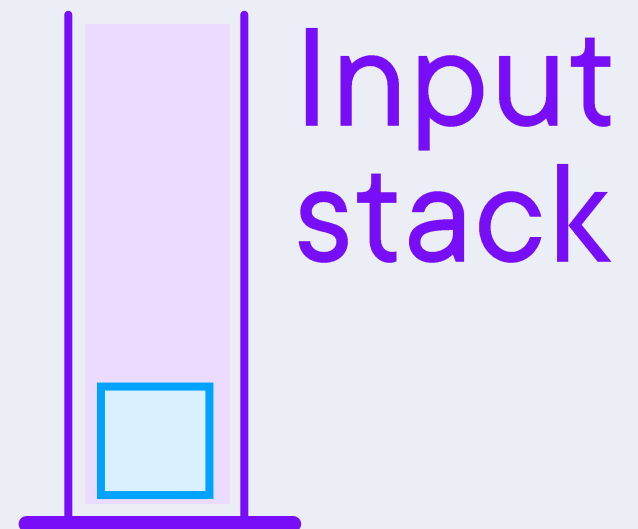
Enqueue

Dequeue



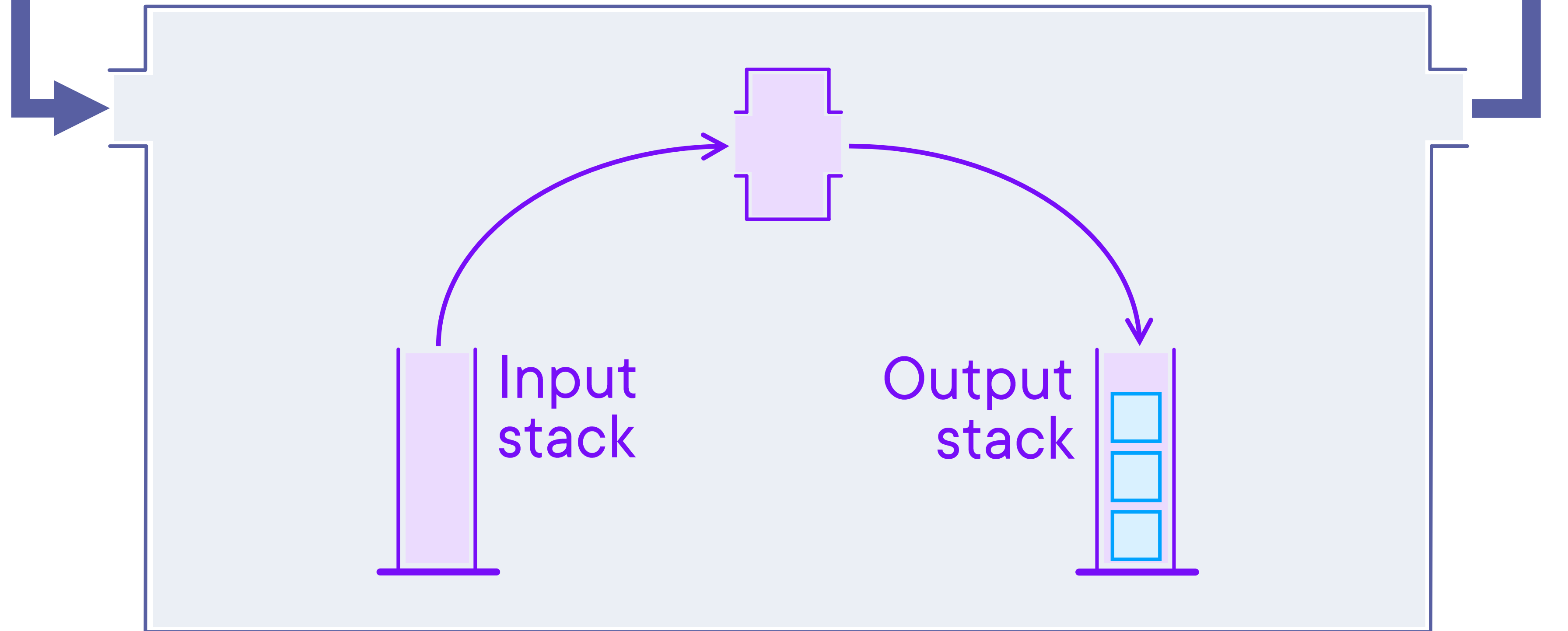
Enqueue

Dequeue



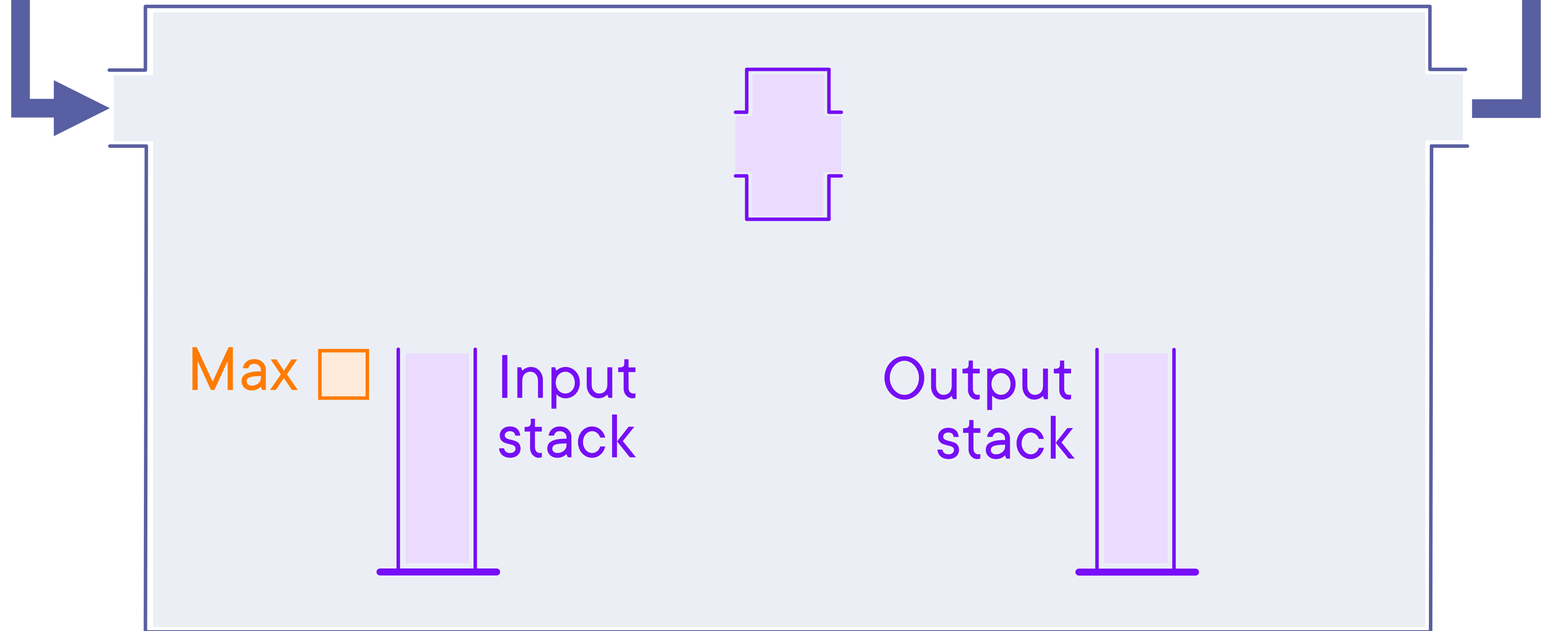
Enqueue

Dequeue



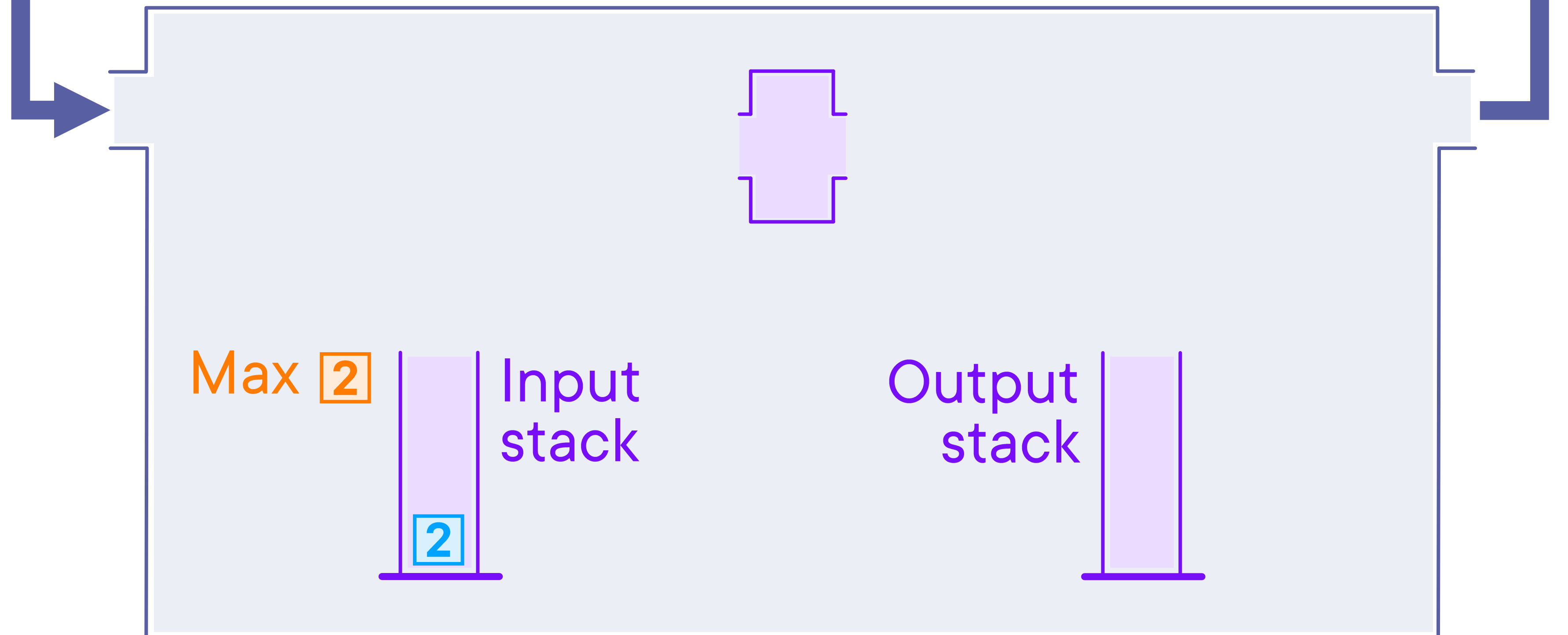
Enqueue

Dequeue



Enqueue

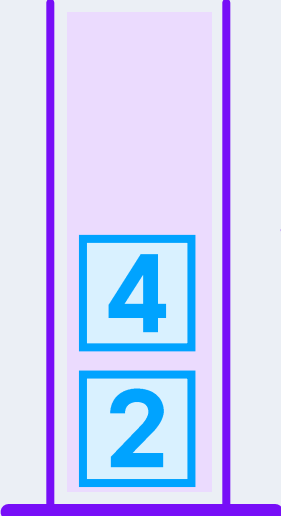
Dequeue



Enqueue

Dequeue

Max **4** Input stack

A vertical stack of two light blue squares. The top square contains the number 4 and the bottom square contains the number 2. The stack is enclosed in a purple outline.

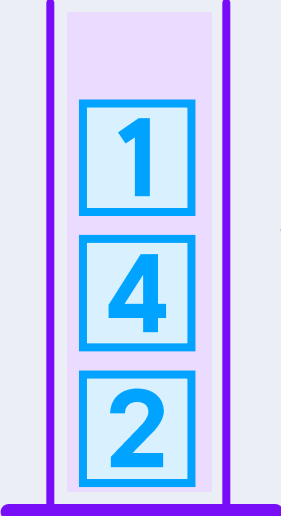
Output stack

An empty vertical stack, represented by a purple outline.

Enqueue

Dequeue

Max **4** Input stack



A vertical stack of three light blue boxes with purple borders. The top box contains the number 1, the middle box contains the number 4, and the bottom box contains the number 2. The stack is labeled 'Input stack' to its right.

Output stack



An empty vertical stack represented by a light purple rectangle with a purple border. It is labeled 'Output stack' to its left.



Enqueue

Dequeue

Max **4**

Input stack

3
1
4
2

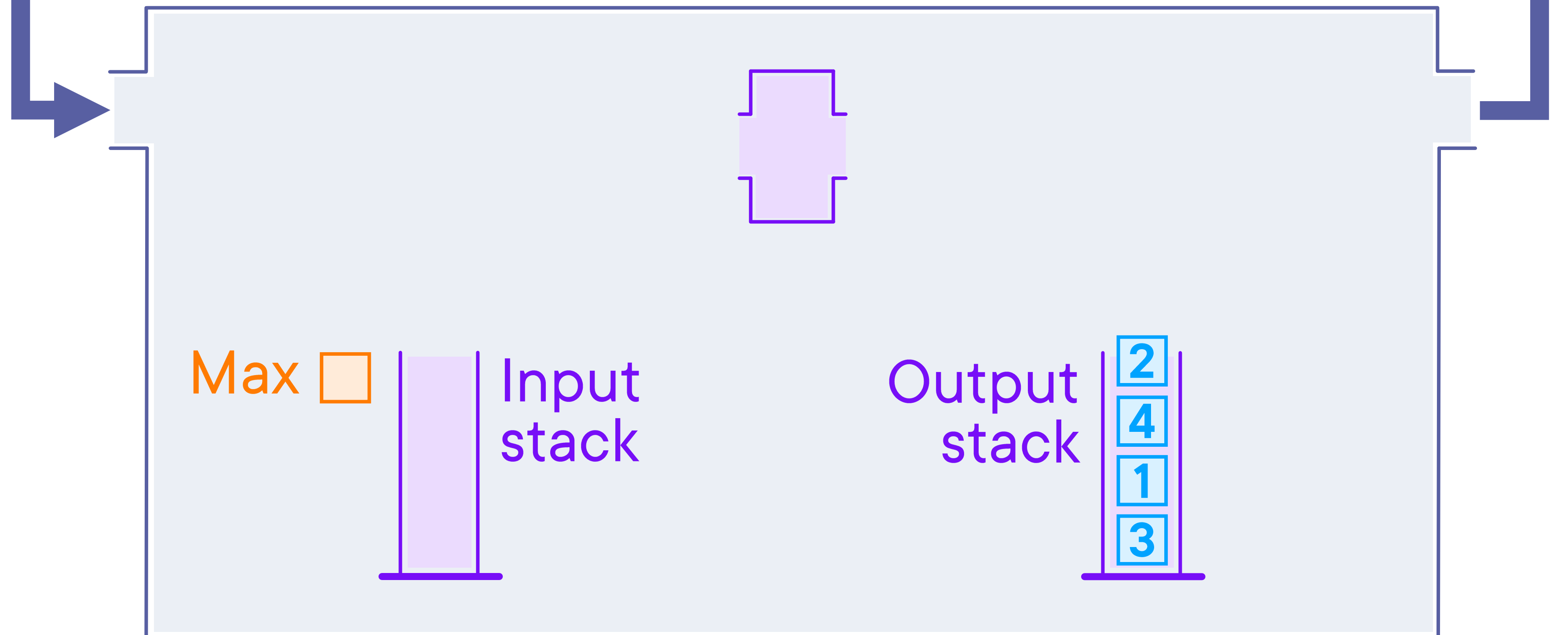
Output stack

--



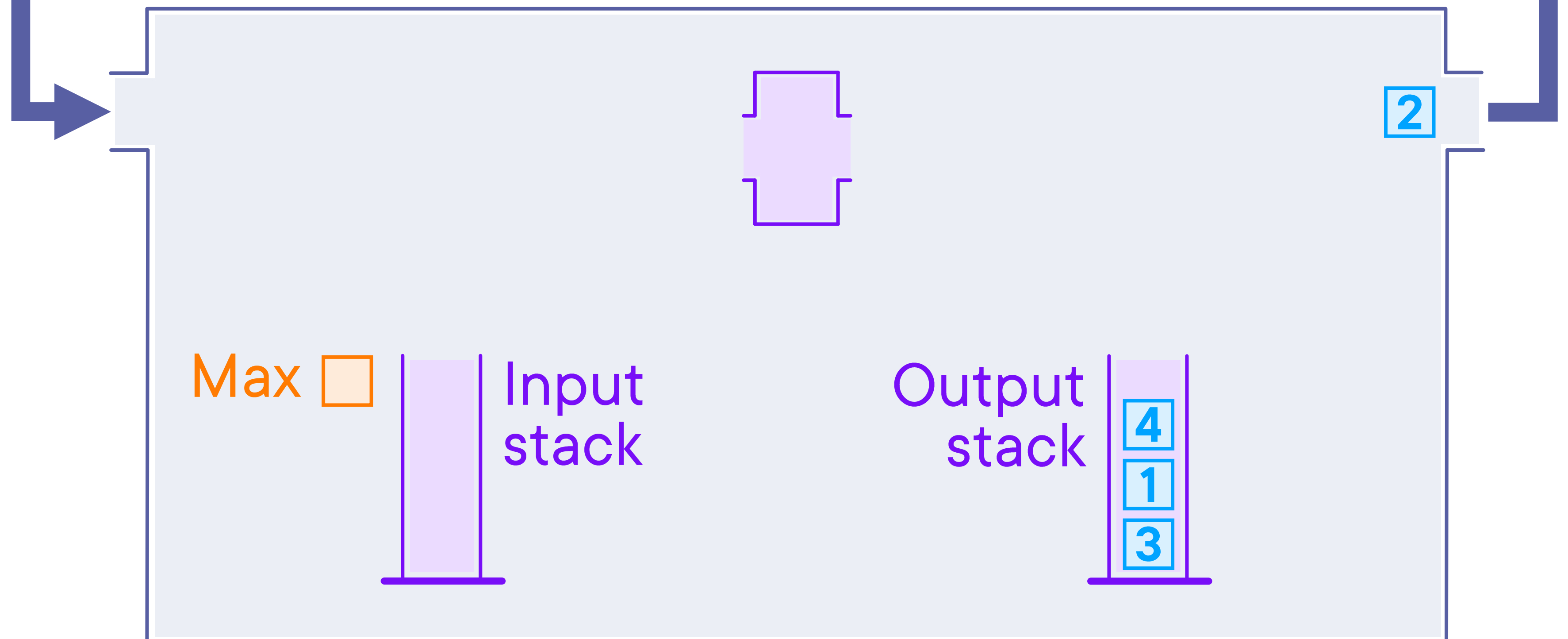
Enqueue

Dequeue



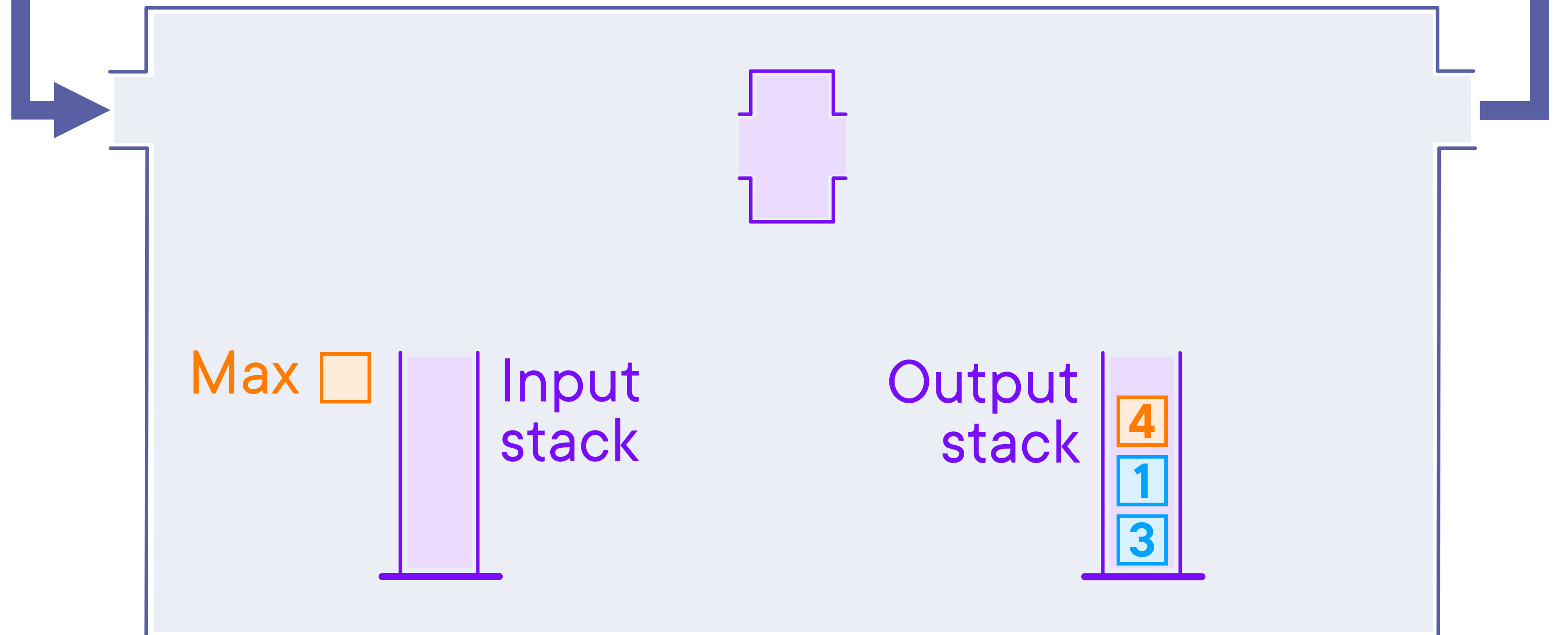
Enqueue

Dequeue



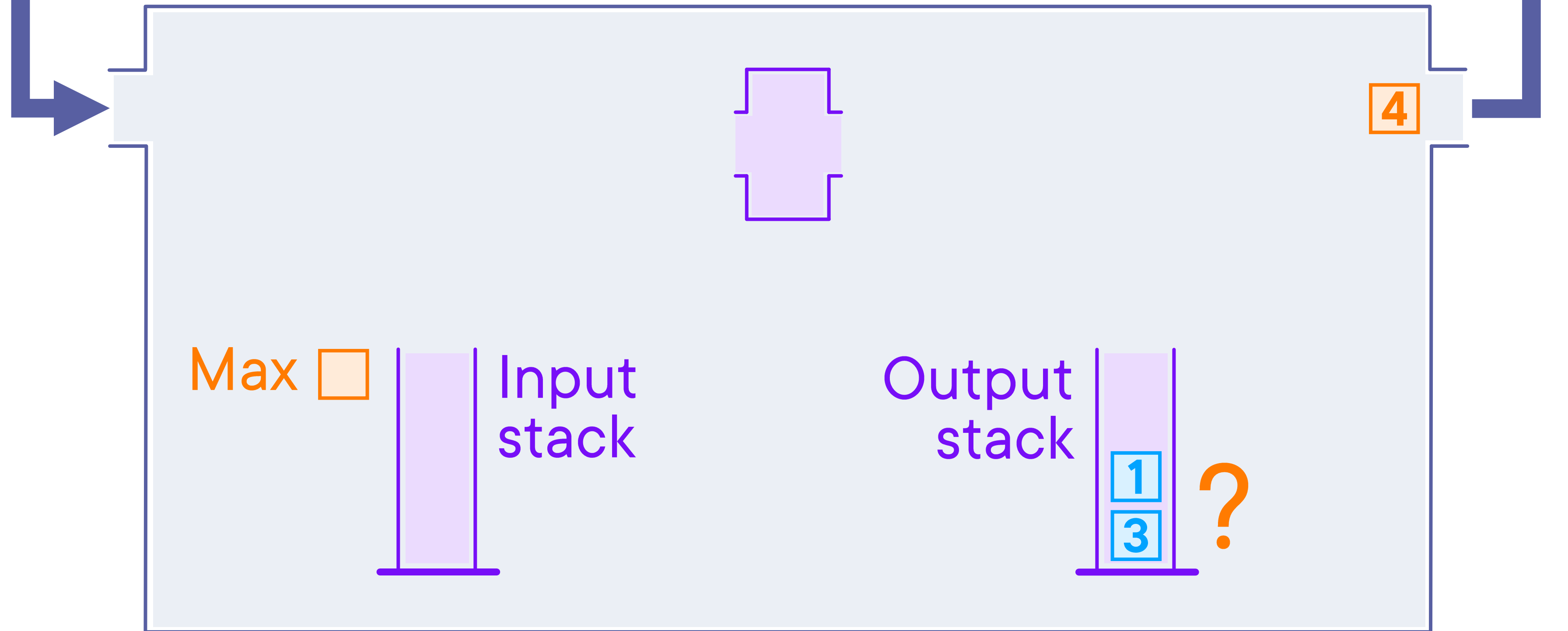
Enqueue

Dequeue



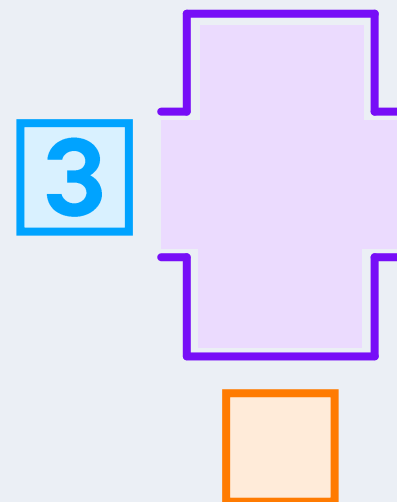
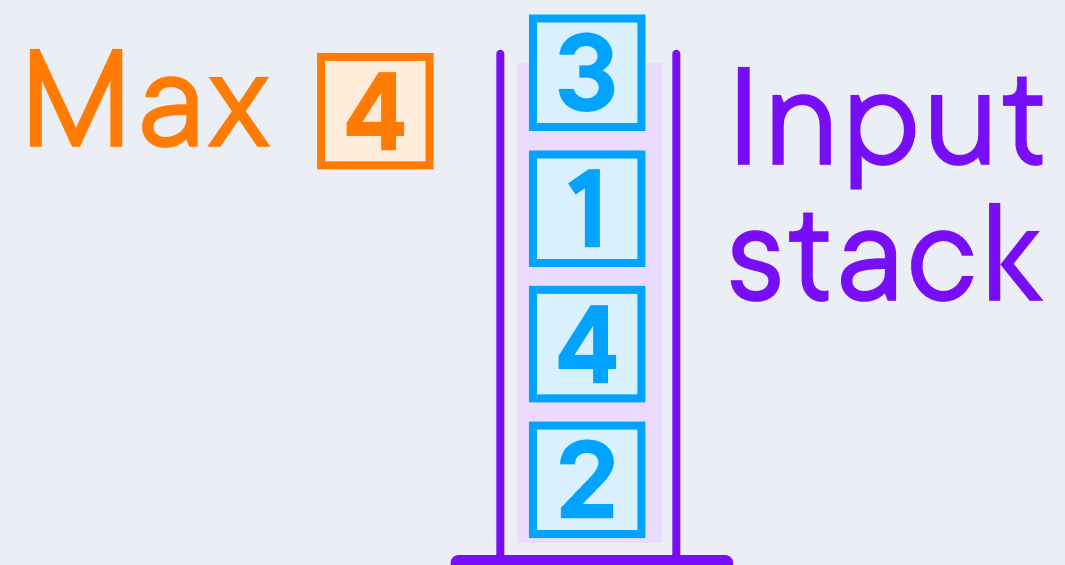
Enqueue

Dequeue



Enqueue

Dequeue



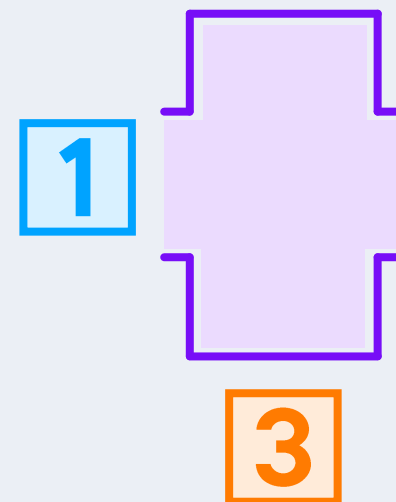
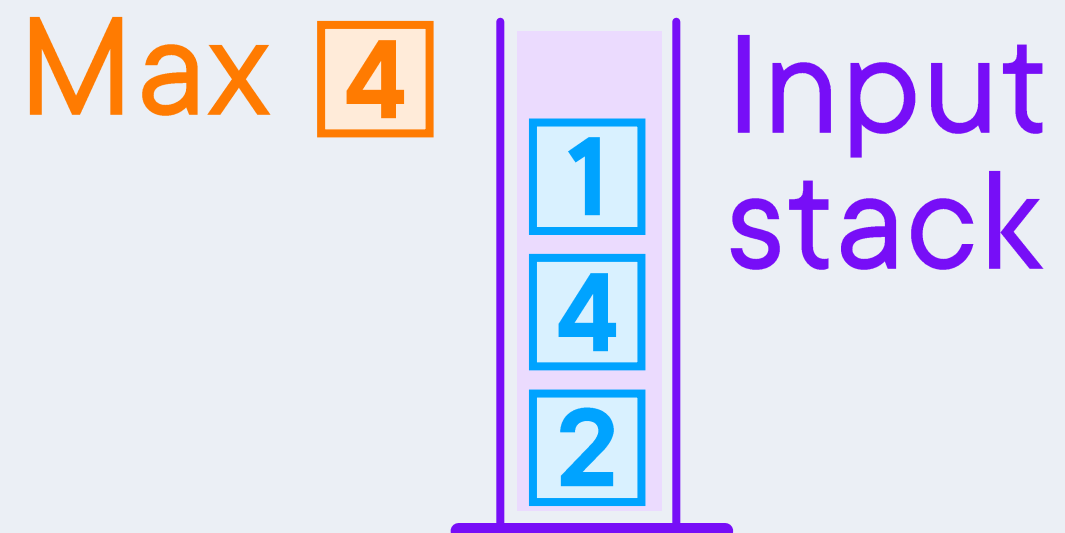
Output stack

The diagram shows a vertical purple rectangular container, which is currently empty.

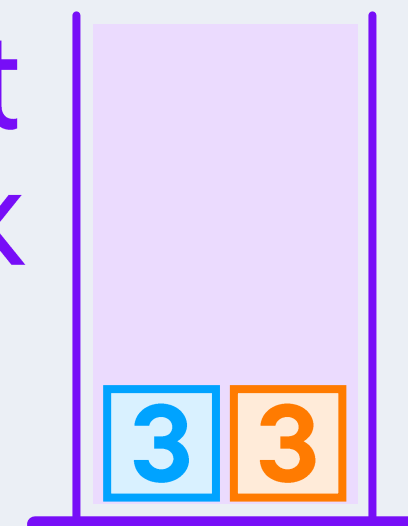


Enqueue

Dequeue

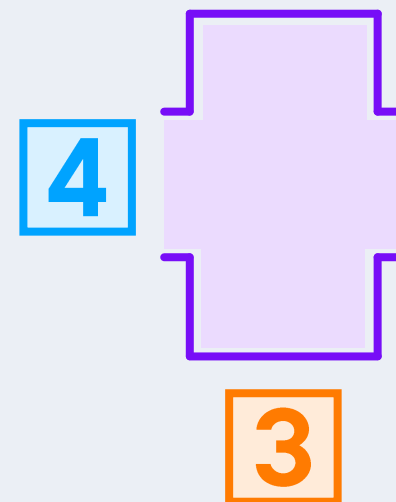
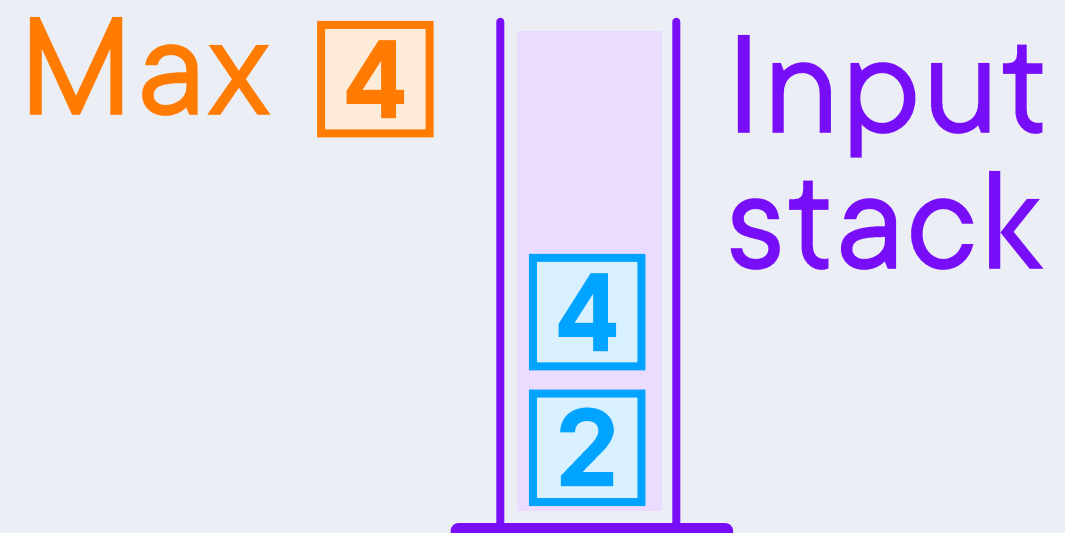


Output stack

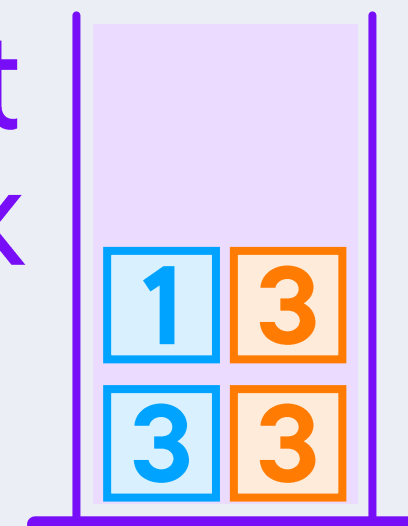


Enqueue

Dequeue



Output stack



Enqueue

Dequeue

Max **4** Input stack

Output stack

2

4

2

4

4

1

3

3

3



Enqueue

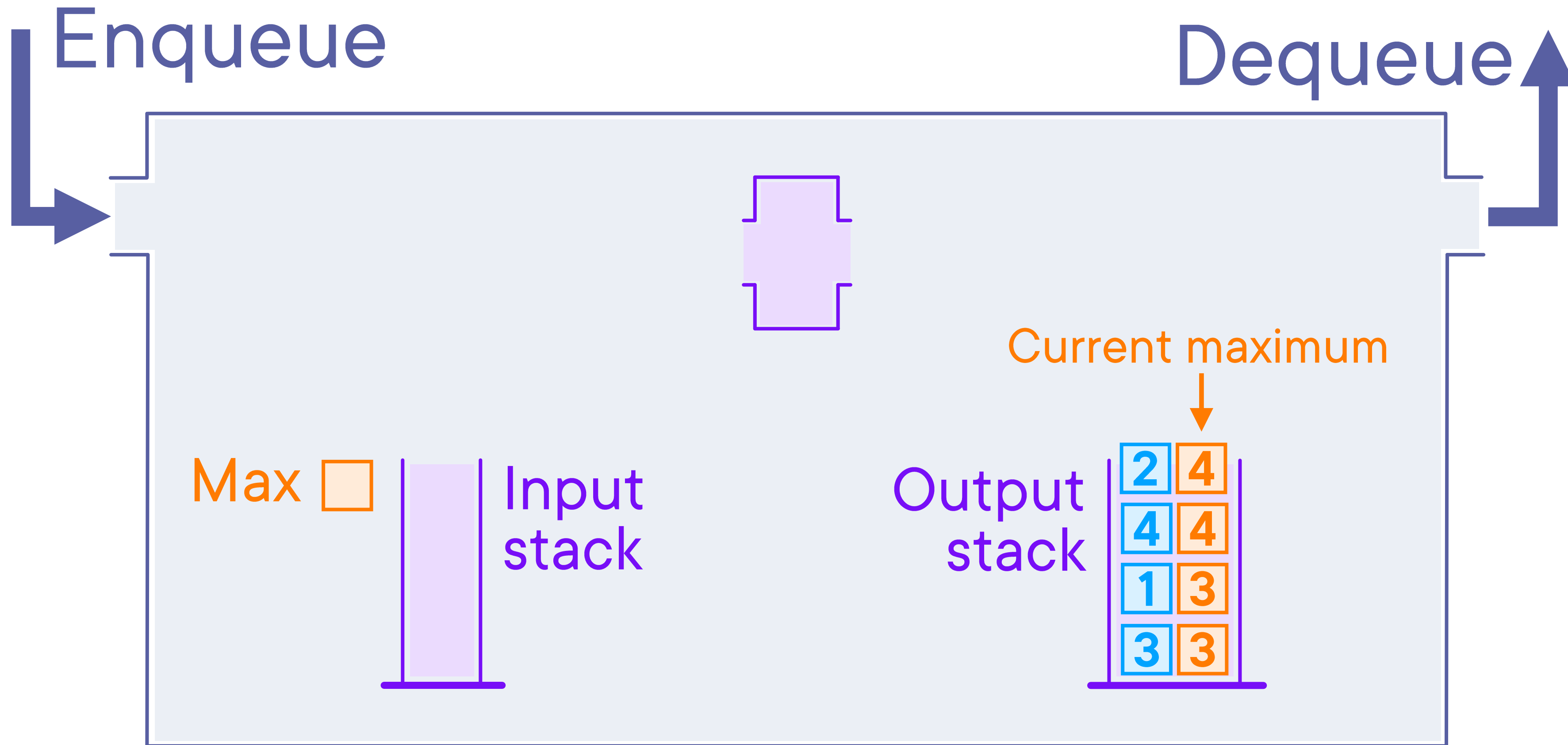
Dequeue

Max 

Input
stack

Current maximum

Output
stack

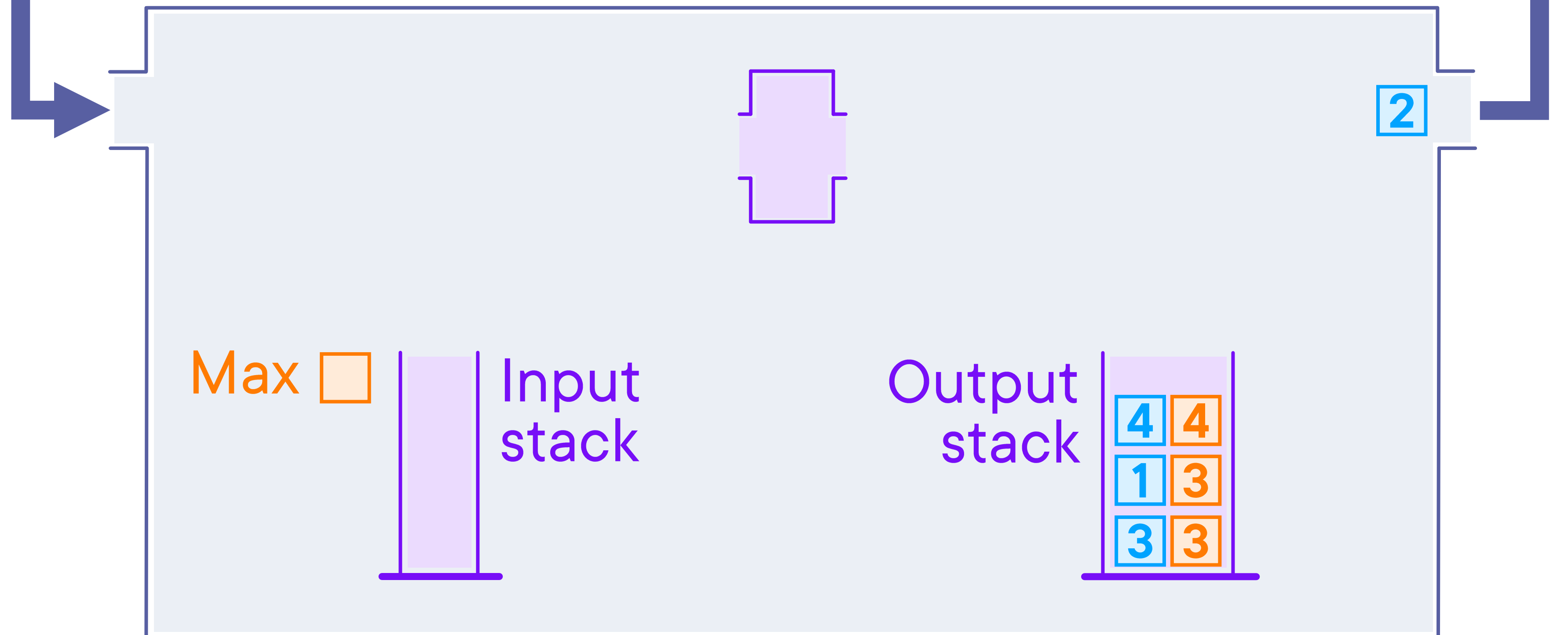


2	4
4	4
1	3
3	3



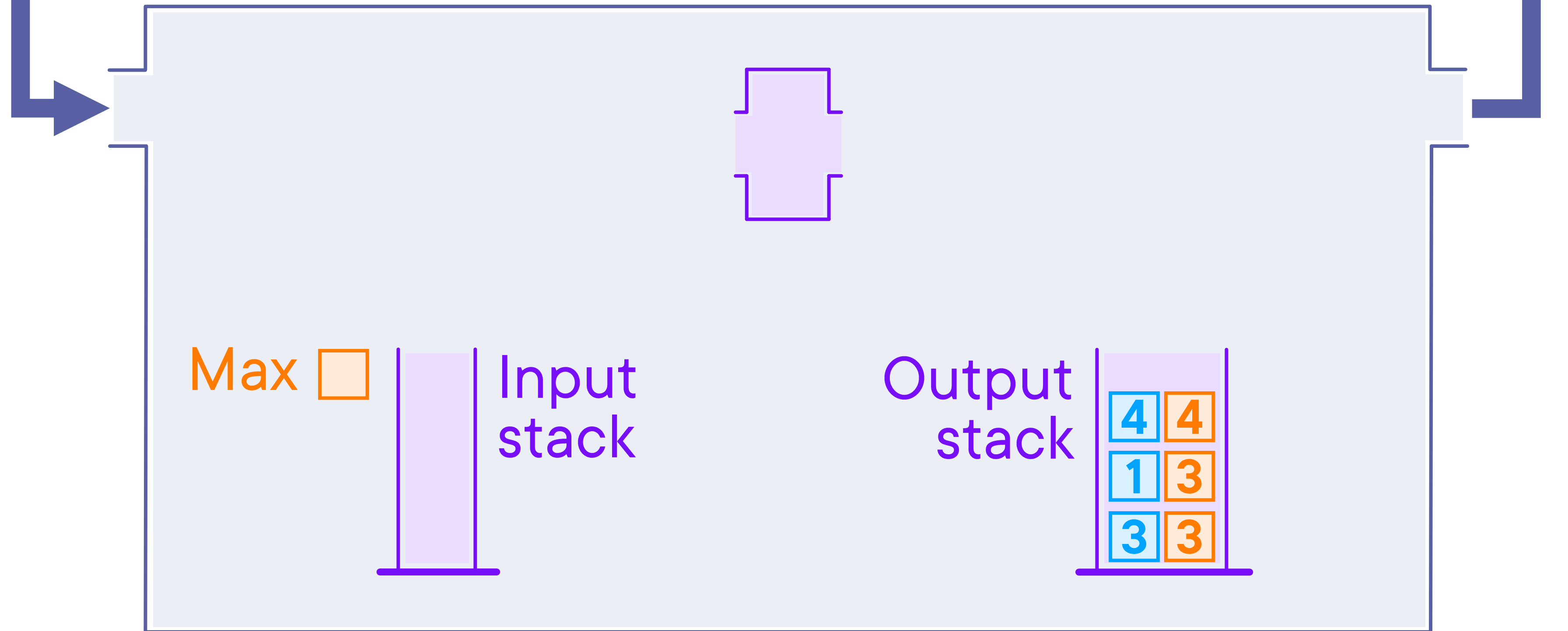
Enqueue

Dequeue



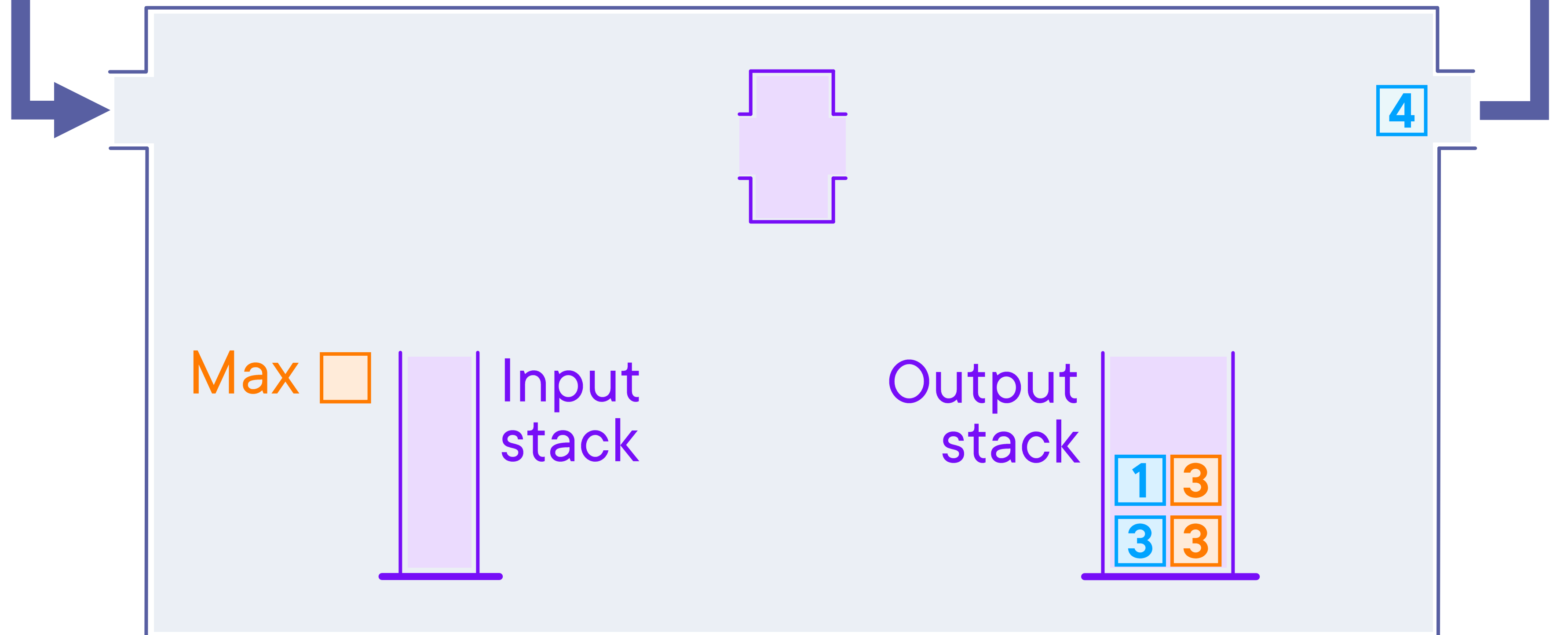
Enqueue

Dequeue



Enqueue

Dequeue



Enqueue

Dequeue

Max 

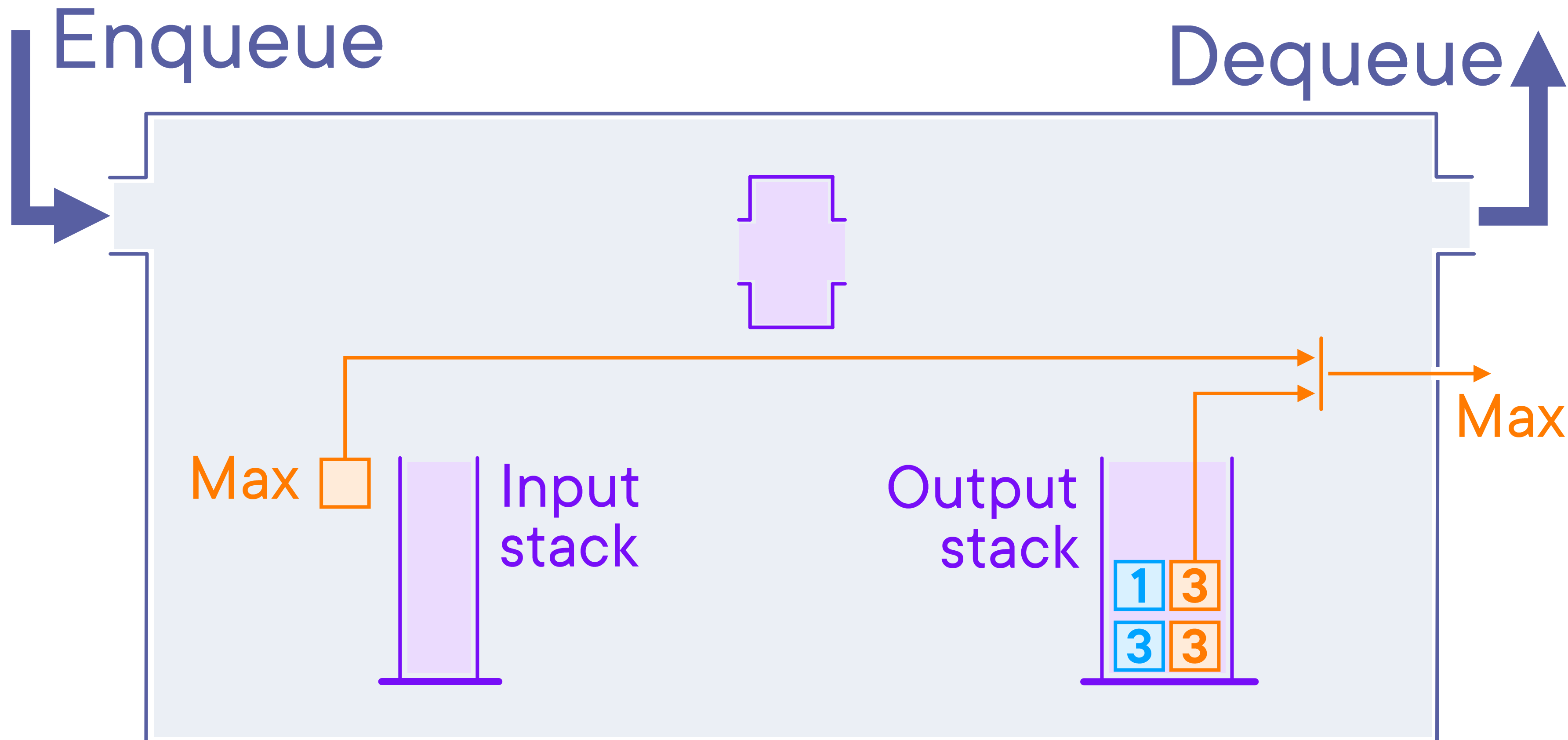
Input
stack

Output
stack

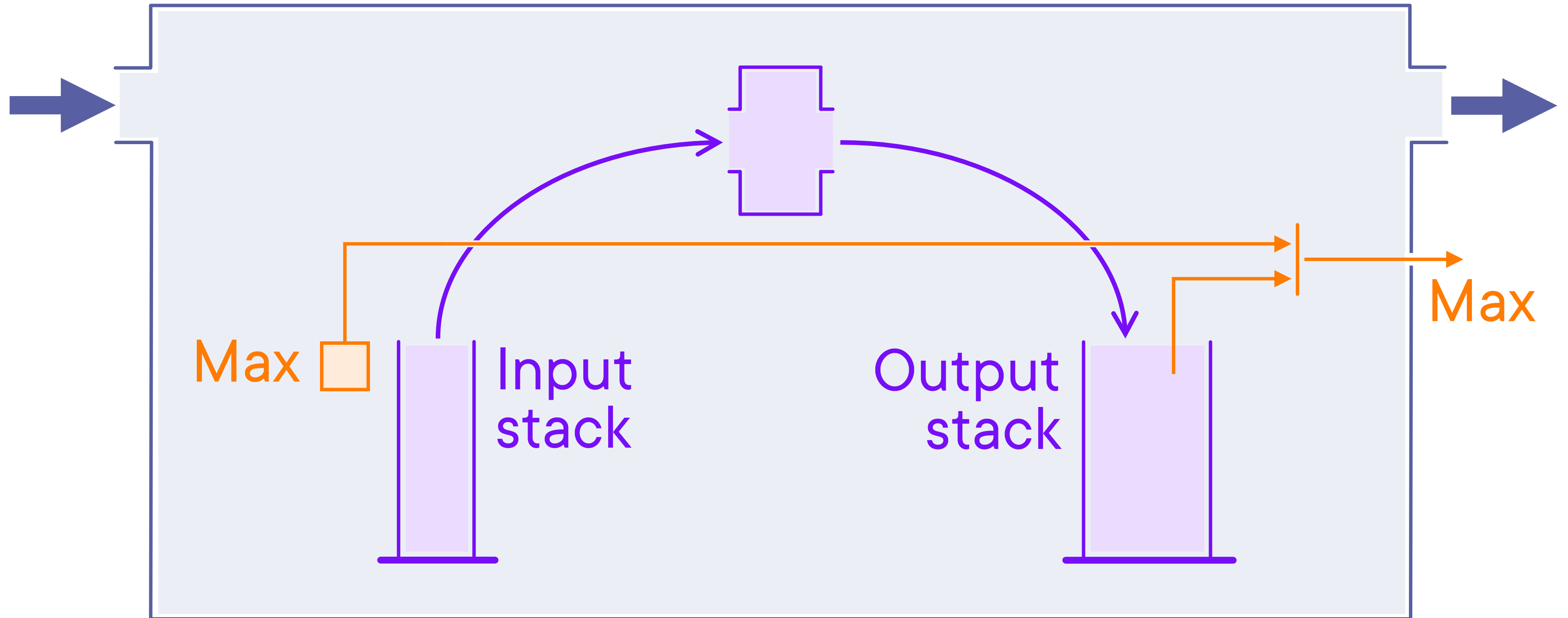
New maximum

1	3
3	3





A maximum queue



Summary



Covered the sliding window technique

- Used a queue to aggregate objects
- A complex analysis running in time proportional to input

Designed a maximum queue

- A queue based on two stacks

Solving a specialized problem

- Look for an appropriate collection design
- Implement the new collection



Summary



Collections applied in this course

- Common array
- List<T>
- SortedList<TKey, TValue>
- LinkedList<T>
- Dictionary<TKey, TValue>
- SortedDictionary<TKey, TValue>
- ImmutableSortedDictionary<TKey, TValue>
- Queue<T>
- Stack<T>
- Custom MaximumQueue<T>

Applied .NET generics through all examples

